Лекція 2. Unit (модульне) тестування в C#

1. Що таке Unit тест?

2. Типи тестування в контексті розробки ПЗ

3. Верифікація та валідація

5. Властивості хорошого Unit тесту

6. Найкращі практики організації тестів

7. Unit Test Frameworks для C#

8. Шаблон AAA (Arrange, Act, Assert)

9. Атрибути для тестування в MS Test

10. Assertion – перевірка результатів

11. Чому розробники не пишуть тести? Розвінчування міфів

12. Користь від модульних тестів

13. Test Driven Development (TDD)

14. Практичні поради

Висновки

Питання для самоперевірки

Додаткові ресурси для вивчення

1. Що таке Unit тест?

1.1 Визначення

Unit тест – це блок коду (зазвичай метод), який викликає блок коду, що тестується, та перевіряє правильність його роботи. Якщо результат тесту не співпадає з очікуваним, то тест вважається непройденим (failure).

Простіше кажучи, Unit тест – це невеликий фрагмент коду, який автоматично перевіряє, чи працює інший фрагмент коду так, як очікується.

1.2 Приклад концепції

Уявіть, що ви написали метод для додавання двох чисел. Unit тест викличе цей метод з конкретними значеннями (наприклад, 10 і 20) і перевірить, чи дорівнює результат очікуваному значенню (30). Якщо так – тест пройдено, якщо ні – виявлено помилку.

2. Типи тестування в контексті розробки ПЗ

2.1 Модульне тестування (Unit Testing)

Тестування кожної атомарної функції програмної системи окремо, з використанням об'єктів штучно змодельованого середовища.

Характеристики:

- Тестується найменша одиниця коду (метод, функція, клас)
- Виконується в ізоляції від інших компонентів
- Використовуються моки та заглушки для імітації залежностей
- Швидко виконується
- Легко локалізувати помилку

2.2 Інтеграційне тестування (Integration Testing)

Тестування, при якому на відповідність вимогам перевіряється інтеграція модулів, їх взаємодія між собою, а також інтеграція підсистем в одну загальну систему.

Характеристики:

- Перевіряється взаємодія між компонентами
- Може включати роботу з базою даних, файловою системою
- Повільніше, ніж модульне тестування
- Складніше локалізувати точку збою

2.3 Системне тестування (System Testing)

Тестування програмного забезпечення повної (зібраної з підсистем) системи з метою перевірки відповідності системи вимогам, як функціональним, так і нефункціональним.

Характеристики:

- Тестується вся система в цілому
- Перевіряється взаємодія всіх компонентів

- Найповільніше виконується
- Найскладніше виявити джерело проблеми

2.4 Проблема інтеграційного тестування

При інтеграційному тестуванні існує багато точок збою, в яких поведінка системи не відповідає очікуваній:

```
Клас, що тестується → Допоміжний клас
    ↓
Бізнес-логіка (точка збою)
    ↓
Доступ до даних (точка збою)
    ↓
Постачальник даних (точка збою)
    ↓
База даних (точка збою)
```

Це значно ускладнює пошук помилок. Модульне тестування вирішує цю проблему, перевіряючи кожен компонент окремо.

3. Верифікація та валідація

3.1 Визначення

Верифікація (Verification) – це процес оцінки системи або її компонентів з метою визначення того, що результати поточного етапу розробки задовольняють умови, сформовані на початку цього етапу.

Валідація (Validation) – це визначення відповідності розроблюваного ПЗ очікуванням та потребам користувача, вимогам до системи. **3.2 Порівняння верифікації та валідації**

Аспект	Верифікація (Verification)	Валідація (Validation)
Головне питання	Чи робимо ми продукт правильно?	Чи робимо ми правильний продукт?
Фокус	Чи реалізована вся функціональність?	Чи правильно реалізована вся функціональність?
Хто проводить	Розробники	Тестувальники

Методи	Інспектування коду, порівняння вимог	Виконання програми
Оцінка	Об'єктивна оцінка реалізованих функцій	Суб'єктивна оцінка програми

Простими словами:

- **Верифікація** перевіряє: "Чи відповідає код технічним специфікаціям?"
- **Валідація** перевіряє: "Чи задовольняє продукт потреби користувача?"

4. Коли потрібно писати Unit тести?

4.1 Коли НЕ потрібно створювати Unit тести

1. Простий код без залежностей

- Тривіальні getter/setter методи
- Прості утилітарні функції
- Код без бізнес-логіки

2. Складний код з великою кількістю залежностей

- Для такого коду слід спочатку провести рефакторінг
- Немає сенсу писати тести для методів, сигнатура яких буде змінюватись
- Краще створювати приймальні тести для такого коду

4.2 Коли ПОТРІБНО створювати Unit тести

3. Складний код без залежностей

- "Заплутана" бізнес-логіка
- Складні алгоритми
- Критичні обчислення

4. Не дуже складний код із залежностями

- Код, що зв'язує компоненти між собою
- Бізнес-логіка середньої складності
- Код, який часто змінюється

5. Властивості хорошого Unit тесту

Якісний Unit тест повинен бути:

5.1 Автоматизованим і повторюваним

Тест має виконуватися автоматично без ручного втручання та давати однаковий результат при кожному запуску.

5.2 Простим у реалізації

Тест має бути легким для написання та розуміння іншими розробниками.

5.3 Постійним

Після написання тест залишається для подальшого використання та регулярного запуску.

5.4 Доступним для всіх

Кожен учасник команди має мати можливість запустити тести на своєму робочому місці.

5.5 Швидким у виконанні

Тест повинен запускатися одним натисненням кнопки та виконуватися швидко (мілісекунди, максимум секунди).

6. Найкращі практики організації тестів

6.1 Розміщення тестів

Загальні правила:

- Тести є частиною системи контролю версій
- Для монолітного застосунку – усі тести розміщують в директорії `tests`
- Для модульного застосунку – тести розміщують в директорії з компонентом, що тестується
- **Завжди розміщайте тести в окремому проекті**

6.2 Іменування проектів

Додавайте до кожного проекту його власний тестовий проект:

Основні проекти:

```
MyApp.Core
MyApp.BusinessLogic MyApp.Web
```

Тестові проекти:

```
MyApp.Core.Tests
MyApp.BusinessLogic.Tests MyApp.Web.Tests
```

6.3 Іменування класів

Правило: Для класу `UserManager` створюється клас з тестами `UserManagerTests`.

Важливо: Кожен клас з тестами повинен містити тести тільки для однієї сутності.

6.4 Іменування методів (Unit тестів)

Загальне правило іменування:

```
[Метод_що_тестується]_[Сценарій]_[Очікувана_поведінка]
```

Приклади:

```
Sum_10plus20_30returned
GetPasswordStrength_AllChars_5Points
ValidateEmail_InvalidFormat_ReturnsFalse
CalculateDiscount_VIPCustomer_Returns20Percent
```

Така назва одразу показує:

- Що саме тестується
- За яких умов
- Який очікується результат

7. Unit Test Frameworks для C#

7.1 Популярні фреймворки

NUnit – www.nunit.org

- Один з найстаріших та найпопулярніших фреймворків
- Багатий функціонал
- Велика спільнота

MS Test – вбудований в Visual Studio

- Інтеграція з Visual Studio
- Підтримка Microsoft
- Простий у використанні для початківців **xUnit.Net** –

https://github.com/xunit/xunit

- Сучасний підхід
- Активна розробка
- Використовується в багатьох open-source проектах

8. Шаблон AAA (Arrange, Act, Assert)

8.1 Суть підходу

Кожен Unit тест складається з трьох чітких фаз:

8.2 Arrange (Підготовка)

Підготовка тестових даних та налаштування середовища. `int`

```
x = 10;

int y = 20; int
expected = 30;
```

8.3 Act (Дія)

Виклик методу, що тестується. `int`

```
actual = Calculator.Add(x, y);
```

8.4 Assert (Перевірка)

Перевірка результату на відповідність очікуваному.

```
Assert.AreEqual(expected, actual);
```

8.5 Повний приклад

```
[TestMethod]
public void Add_10plus20_30returned()
{
    // Arrange     int
x = 10;     int y =
20;     int expected =
30;

// Act
    int actual = Calculator.Add(x, y);

    // Assert
    Assert.AreEqual(expected, actual);
}
```

9. Атрибути для тестування в MS Test

9.1 Основні атрибути класів

[TestClass] – позначає клас, що містить тести

```
[TestClass]
public class CalculatorTests
{
    // тести тут }
```

9.2 Атрибути методів

[TestMethod] – позначає окремий модульний тест

```
[TestMethod]
public void Add_TwoNumbers_ReturnsSum()
```

```
{
    // код тесту
}
```

[TestInitialize] – метод для ініціалізації, викликається перед КОЖНИМ тестом

```
[TestInitialize] public
void Setup()
{
    // підготовка для кожного тесту }
```

[TestCleanup] – метод для звільнення ресурсів, викликається після КОЖНОГО тесту

```
[TestCleanup] public
void Cleanup()
{
    // очищення після кожного тесту }
```

9.3 Атрибути рівня класу

[ClassInitialize] – викликається ОДИН РАЗ для всього класу з тестами, перед запуском першого тесту

```
[ClassInitialize]
public static void ClassSetup(TestContext context)
{
    // ініціалізація для всього класу
}
```

[ClassCleanup] – викликається ОДИН РАЗ для всього класу після завершення всіх тестів

```
[ClassCleanup]
public static void ClassTeardown()
{
    // очищення після всіх тестів класу
}
```

9.4 Атрибути рівня збірки

[AssemblyInitialize] – викликається перед тим, як почнуть працювати тести в збірці

[AssemblyCleanup] – викликається після того, як відпрацюють усі тести в збірці

10. Assertion – перевірка результатів

10.1 Типи класів для перевірок

Assert – основний клас для перевірок

- Порівняння двох вхідних значень
- Багато перевантажень для порівняння різних типів даних

CollectionAssert – для роботи з колекціями

- Порівняння двох колекцій
- Перевірка елементів колекції

StringAssert – для роботи з рядками

- Порівняння рядків
- Перевірка підрядків, регулярних виразів

10.2 Основні методи класу Assert

Assert.AreEqual(expected, actual) Перевіряє рівність двох елементів за значенням.

```
Assert.AreEqual(30, calculator.Add(10, 20));
```

Assert.AreSame(expected, actual) Перевіряє, чи посилаються два об'єкти на один і той же екземпляр.

```
Assert.AreSame(instance1, instance2);
```

Assert.IsInstanceOfType(object, type) Перевіряє тип об'єкта.

```
Assert.IsInstanceOfType(result, typeof(Customer));
```

Assert.IsTrue(condition) Перевіряє істинність логічного виразу.

```
Assert.IsTrue(user.IsActive);
```

Assert.IsFalse(condition) Перевіряє хибність логічного виразу.

```
Assert.IsFalse(user.IsBlocked);
```

Assert.IsNull(object) Перевіряє, що об'єкт є null.

```
Assert.IsNull(deletedUser);
```

Assert.IsNotNull(object) Перевіряє, що об'єкт не є null.

```
Assert.IsNotNull(createdUser);
```

11. Чому розробники не пишуть тести? Розвінчування міфів

11.1 Міф 1: "Написання тестів займає надто багато часу"

Реальність:

- Тести створюються разом з кодом, а не в кінці
- Час на написання тесту часто менший за час на ручну перевірку
- Тести економлять час у довгостроковій перспективі, запобігаючи помилкам

Питання: Як ви зараз перевіряєте, що код працює? Запускаєте вручну? Чому б не витратити цей же час на написання автоматизованого Unit тесту? **11.2 Міф 2: "Запуск Unit тестів займає надто багато часу"**

Реальність:

- Unit тести повинні виконуватися швидко (мілісекунди) • Повільні тести – це ознака інтеграційних тестів • Має бути кілька рівнів тестування:
  - Unit тести – швидко і часто ○ Інтеграційні тести – рідше, але регулярно **11.3 Міф 3: "Це не моя робота – тестувати код"**

Реальність:

- Наше завдання – розробляти ПРАЦЮЮЧЕ ПЗ
- Щоб заявляти, що код працює, потрібні докази
- Тести – це професійна відповідальність розробника

11.4 Міф 4: "Я не можу тестувати код, бо не знаю точно, як він має працювати"

Реальність:

- Якщо ви не знаєте, як має працювати код, як ви могли почати його писати?
- Розуміння очікуваної поведінки – передумова для написання коду
- Тести допомагають формалізувати вимоги

12. Користь від модульних тестів

12.1 Інструмент боротьби з інверсією в коді

Тести виявляють проблеми в архітектурі та дизайні коду на ранніх етапах.

12.2 Інвестиція в якісну архітектуру

Ізоляція залежностей:

- Змушує розробника думати про абстракції
- Сприяє використанню принципів SOLID
- Робить код більш модульним

Природний рефакторінг:

- Розробник, який знає, що код буде використовуватись в тестах, природно створює якісніший дизайн
- Стимулює рефакторінг при перших ознаках сильної зв'язності
- Підвищує загальну якість архітектури

12.3 Додаткові переваги

- **Документація:** Тести показують, як використовувати код
- **Впевненість при змінах:** Можна безпечно рефакторити, маючи тести
- **Швидкий зворотний зв'язок:** Миттєве виявлення регресій
- **Спрощення дебагінгу:** Легше знайти джерело проблеми

13. Test Driven Development (TDD)

13.1 Що таке TDD?

Test-Driven Development (TDD) – розробка через тестування. Підхід розробки ПЗ, який полягає в написанні Unit тесту ПЕРЕД написанням самого коду.

13.2 Традиційний підхід vs TDD

Традиційний спосіб:

1. Пишемо метод/клас
2. Пишемо Unit тест
3. Виконуємо Unit тест
4. Виправляємо помилки
5. Повторюємо кроки 3-4

TDD підхід:

1. Пишемо Unit тест (тест не проходить – RED)
2. Пишемо мінімальний код для проходження тесту
3. Тест проходить (GREEN)
4. Рефакторінг коду (REFACTOR)
5. Повторюємо цикл **13.3**

Цикл Red-Green-Refactor

RED (Червоний):

- Напишіть тест, який не проходить
- Тест описує очікувану функціональність **GREEN (Зелений):**

- Напишіть мінімальний код, щоб тест пройшов
- Не турбуйтесь про оптимізацію

REFACTOR (Рефакторінг):

- Покращте код, зберігаючи функціональність
- Тести гарантують, що нічого не зламалось

13.4 Переваги TDD

Якість дизайну:

- Код автоматично стає більш тестованим
- Природне використання абстракцій
- Слабка зв'язність компонентів

Покриття тестами:

- 100% покриття критичної функціональності
- Тести написані до коду, а не після **Впевненість:**

- Код працює з моменту написання
- Безпечний рефакторінг

Документація:

- Тести показують, як використовувати код
- Актуальна документація, що виконується

13.5 Коли використовувати TDD?

Рекомендується:

- Для нової функціональності з чіткими вимогами
- Для складної бізнес-логіки
- Для критичних компонентів системи
- Коли команда має досвід TDD

Може бути складно:

- При прототипуванні (вимоги часто змінюються)
- Для UI-компонентів
- При роботі зі сторонніми API, поведінка яких невідома
- Для початківців без досвіду тестування

14. Практичні поради

14.1 Що робити прямо зараз

1. **Почніть з малого** – напишіть тест для одного простого методу
2. **Використовуйте AAA** – структуруйте всі тести однаково
3. **Дотримуйтесь конвенцій** – називайте тести зрозуміло
4. **Запускайте регулярно** – інтегруйте в процес розробки
5. **Навчайтесь на прикладах** – вивчайте тести з open-source проектів

14.2 Ознаки поганого тесту

- Тест залежить від інших тестів
- Тест залежить від порядку виконання
- Тест працює з реальною базою даних чи файловою системою
- Тест виконується довго (більше секунди)
- Тест перевіряє кілька різних речей
- Назва тесту незрозуміла **14.3**

Метрики якості тестування

Code Coverage (Покриття коду):

- Відсоток коду, покритого тестами
- Високе покриття ≠ якісні тести
- Рекомендоване покриття: 70-80% для критичної логіки **Mutation Testing:**

- Внесення штучних змін у код
- Перевірка, чи виявлять тести ці зміни **Test Maintenance:**

- Час на підтримку тестів
- Частота помилкових спрацювань

Висновки

Модульне тестування – це не просто технічна практика, а філософія розробки якісного програмного забезпечення. Основні тези, які варто запам'ятати:

1. **Unit тести перевіряють найменші одиниці коду в ізоляції** – це дозволяє швидко виявляти помилки.
2. **Хороші тести – автоматизовані, швидкі, незалежні та зрозумілі** – дотримуйтесь цих принципів.
3. **Шаблон AAA (Arrange-Act-Assert)** – структурує тести та робить їх читабельними.
4. **Тести – це інвестиція** – вони економлять час у довгостроковій перспективі та покращують архітектуру.
5. **TDD – потужний підхід** – написання тестів перед кодом призводить до кращого дизайну.
6. **Почніть писати тести вже сьогодні** – навіть прості тести краще, ніж їх відсутність.

Пам'ятайте: професійний розробник пише тести. Це не додаткова робота, а невід'ємна частина процесу створення якісного програмного забезпечення.

Питання для самоперевірки

1. Чим відрізняється Unit тестування від інтеграційного?
2. Що таке верифікація і валідація? Як вони співвідносяться?
3. Який код варто покривати Unit тестами, а який – ні?
4. Розкажіть про шаблон AAA. Наведіть приклад.
5. Які атрибути використовуються для позначення тестів у MS Test?
6. Які основні методи класу Assert ви знаєте?
7. У чому суть підходу TDD?
8. Назвіть п'ять властивостей хорошого Unit тесту.
9. Як правильно іменувати тестові методи?
10. Чому тести є інвестицією в якісну архітектуру?

Додаткові ресурси для вивчення

- **Книги:**
    - "The Art of Unit Testing" – Roy Osherove ○ "Test Driven Development: By Example" – Kent Beck